

Godfrey-249 / Phase_5_Project

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

This is the repository of Group 2 phase 5 project

☆ 0 stars

🍴 0 forks

👁 0 watching

🔗 Branches

📈 Activity

🏷 Tags

🌐 Public repository

🔗

🔗 2 Branches

🏷 0 Tags

🔗

📁

🔍 Go to file

t

Go to file

+

Add file

Code

...

god249

Added Data Report

15ec30b · 1 hour ago

📁 catboost_info	Made changes	5 days ago
📄 .DS_Store	Update notebooks	2 weeks ago
📄 .gitignore	modified gitignore	2 weeks ago
📄 Final_Data.xlsx	Made changes	5 days ago
📄 Mental Health Report.pptx	Added report	last week
📄 Mental_Health_CRISP_DM_Rep...	Added Data Report	1 hour ago
📄 Mental_Health_Classifier.ipynb	Made changes	5 days ago
📄 Mental_Health_Identifier2.ipynb	Made changes	5 days ago
📄 README.md	Update README.md	2 weeks ago
📄 Tableau Analysis mental health...	ADD:Tableau Visualizations Dashboard	last week
📄 api.py	Added api.py	5 days ago
📄 app.py	Added api.py	5 days ago
📄 distribution_data.csv	Added Files	2 weeks ago
📄 requirements.txt	Add/update dependencies	5 days ago

📖 README

Mental Health Prediction Assistant

A Streamlit web app that predicts potential mental health disorders and severity using:

- **Signs-Based Input** (Natural Language)
- **Lifestyle-Based Input** (Structured Data)

This assistant helps screen mental health concerns and provide personalized self-care or professional suggestions.

Features

◆ Signs-Based Prediction (NLP)

- Enter free-text symptoms (e.g., "I feel anxious and can't sleep").
- App uses embeddings & cosine similarity to predict possible disorders.
- Shows top 3 disorders and self-care/professional recommendations.

◆ Lifestyle-Based Prediction (Structured)

- Fill a lifestyle survey (sleep, diet, stress, etc.).
- Predicts whether you're at risk, and estimates the **severity level**.

Project Structure

```
phase_5_project/
├── app.py           # Streamlit app code
├── model_nlp.pkl    # Pickled NLP model + embeddings
├── model_structured.pkl # Pickled structured models & encoders
├── requirements.txt # Required Python libraries
└── README.md       # This documentation
```



How to Run the App

1. Clone the Repo

```
git clone https://github.com/Godfrey-249/Phase_5_Project.git
cd Phase_5_Project
```



2. Install Dependencies

```
pip install -r requirements.txt
```



3. Launch the App

```
streamlit run app.py
```



Screenshot Placeholder

You can include a screenshot using:

```
st.image("screenshot.png")
```



Full App Code

```
import streamlit as st
import pickle
import numpy as np
import pandas as pd
from sentence_transformers import SentenceTransformer, util

# Load NLP model bundle
with open("model_nlp.pkl", "rb") as f:
    nlp_bundle = pickle.load(f)

nlp_model = nlp_bundle["model"]
disorder_embeddings = nlp_bundle["embeddings"]
signs_dict = nlp_bundle["signs_dict"]
df_recs = nlp_bundle["recommendations"]

# Load Structured model bundle
with open("model_structured.pkl", "rb") as f:
    structured_bundle = pickle.load(f)

risk_model = structured_bundle["risk_model"]
severity_model = structured_bundle["severity_model"]
encoders = structured_bundle["encoders"]

st.title(" Mental Health Prediction Assistant")
st.markdown("Choose a mode below to begin:")

mode = st.sidebar.radio("Choose Prediction Mode", ["Signs-Based", "Lifestyle-Based"])

if mode == "Signs-Based":
    st.header("Signs-based Prediction")
    user_input = st.text_area("Describe your signs (e.g., 'I feel anxious and overwhelmed'):")

    if st.button("Predict Disorder"):
        input_embed = nlp_model.encode(user_input)
        similarities = {}

        for disorder, embed in disorder_embeddings.items():
            score = util.cos_sim(input_embed, embed).item()
            similarities[disorder] = score

        sorted_disorders = sorted(similarities.items(), key=lambda x: x[1], reverse=True)[:3]

        st.subheader("Predicted Disorders")
        for disorder, score in sorted_disorders:
            st.write(f"- **{disorder}** (Score: {score:.2f})")
```



```

top_disorder = sorted_disorders[0][0]

rec_row = df_recs[df_recs['Disorder'].str.lower() == top_disorder.lower()]
if not rec_row.empty:
    st.subheader("Recommendations")
    self_recs = rec_row['Reccomendations; Self'].dropna().values
    prof_recs = rec_row['Reccomendation 2; Proffesional'].dropna().values
    other_recs = rec_row['Other Reccomendation'].dropna().values

    st.markdown("***Self-care:**")
    for i, val in enumerate(self_recs[:3], 1):
        st.markdown(f"- {val}")

    st.markdown("***Professional Help:**")
    for i, val in enumerate(prof_recs[:2], 1):
        st.markdown(f"- {val}")

    st.markdown("***Other:**")
    for i, val in enumerate(other_recs[:2], 1):
        st.markdown(f"- {val}")
else:
    st.error("No recommendations found.")

elif mode == "Lifestyle-Based":
    st.header("Lifestyle-based Prediction")

    age = st.slider("Age", 10, 100, 25)
    gender = st.selectbox("Gender", encoders["Gender"].classes_)
    occupation = st.selectbox("Occupation", encoders["Occupation"].classes_)
    consultation = st.selectbox("Consultation History", encoders["Consultation_History"].classes_)
    stress = st.selectbox("Stress Level", encoders["Stress_Level"].classes_)
    sleep = st.slider("Sleep Hours", 0, 12, 6)
    work = st.slider("Work Hours", 0, 16, 8)
    physical = st.slider("Physical Activity (hrs/week)", 0, 14, 3)
    social = st.slider("Social Media Usage (hrs/day)", 0, 12, 4)
    diet = st.selectbox("Diet Quality", encoders["Diet_Quality"].classes_)
    smoking = st.selectbox("Smoking Habit", encoders["Smoking_Habit"].classes_)
    alcohol = st.selectbox("Alcohol Consumption", encoders["Alcohol_Consumption"].classes_)
    medication = st.selectbox("Medication Usage", encoders["Medication_Usage"].classes_)

    if st.button("Assess Risk"):
        sample = pd.DataFrame([
            'Age': age,
            'Gender': encoders['Gender'].transform([gender])[0],
            'Occupation': encoders['Occupation'].transform([occupation])[0],
            'Consultation_History': encoders['Consultation_History'].transform([consultation])[0],
            'Stress_Level': encoders['Stress_Level'].transform([stress])[0],
            'Sleep_Hours': sleep,
            'Work_Hours': work,
            'Physical_Activity_Hours': physical,
            'Social_Media_Usage': social,
            'Diet_Quality': encoders['Diet_Quality'].transform([diet])[0],
            'Smoking_Habit': encoders['Smoking_Habit'].transform([smoking])[0],
            'Alcohol_Consumption': encoders['Alcohol_Consumption'].transform([alcohol])[0],
            'Medication_Usage': encoders['Medication_Usage'].transform([medication])[0]
        ])

        prediction = risk_model.predict(sample)[0]
        label = "Low Risk (No Disorder)" if prediction == 0 else "High Risk (Disorder Likely)"

```

```
st.markdown(f"### Prediction: **{label}**")

if prediction == 1:
    severity = severity_model.predict(sample)[0]
    st.markdown(f"### Estimated Severity: **{severity}**")
```



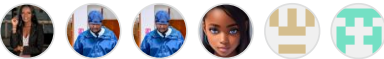
Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 6



Languages

Jupyter Notebook 99.6% Python 0.4%

Suggested workflows

Based on your tech stack

Python application

Create and test a Python application.

Configure

Publish Python Package

Publish a Python Package to PyPI on release.

Configure

Django

Build and Test a Django Project

Configure